📖 BankStart MVP — Подробные объяснения

Обучающий материал по архитектурным решениям и паттернам

📋 Оглавление

1. Почему Spring Boot?
2. Паттерн Layered Architecture
3. JPA и ORM
4. Паттерн DTO
5. Dependency Injection
6. Thymeleaf и шаблоны
7. Метод Monte Carlo
8. BigDecimal для денег
9. Bean Validation
10. Профили и конфигурация

---

1. Почему Spring Boot?

Проблема

Настройка Java Enterprise приложения требовала:
- XML конфигурации (сотни строк)
- Ручного управления зависимостями
- Настройки сервера приложений (Tomcat, JBoss)

Решение Spring Boot

```
@SpringBootApplication  // <-- Одна аннотация!
public class BankStartApplication {
    public static void main(String[] args) {
        SpringApplication.run(BankStartApplication.class, args);
    }
}
```

Что делает @SpringBootApplication?

```
@SpringBootApplication = @Configuration
                       + @EnableAutoConfiguration
                       + @ComponentScan
```

Аннотация	Что делает
`@Configuration`	Класс — источник bean definitions
`@EnableAutoConfiguration`	Автонастройка на основе classpath
`@ComponentScan`	Сканирование компонентов в пакете

Автоконфигурация в действии

```
Видит spring-boot-starter-web → Настраивает Tomcat
Видит spring-boot-starter-data-jpa → Настраивает Hibernate
Видит h2 в classpath → Создаёт in-memory БД
```

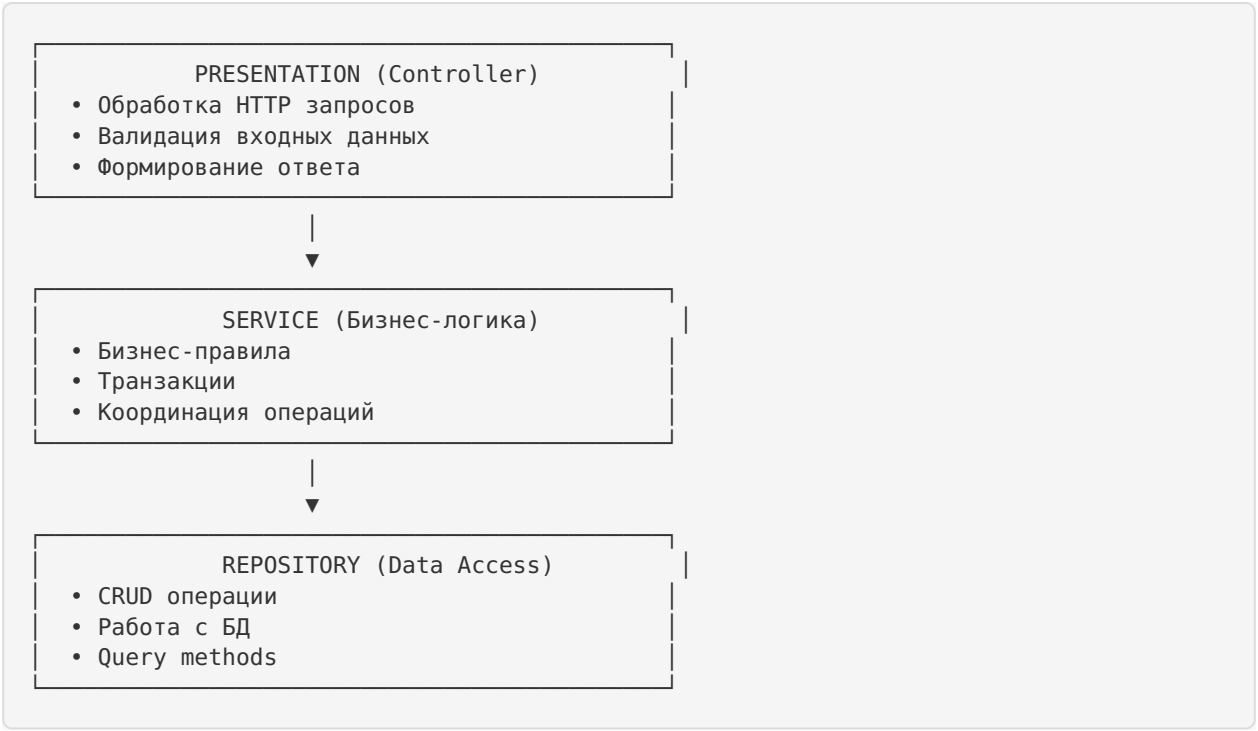
---

2. Паттерн Layered Architecture

Что это?

Разделение приложения на горизонтальные слои, где каждый слой:
- Имеет свою ответственность
- Взаимодействует только с соседними слоями
- Может быть заменён без изменения других

Слои в BankStart



Пример потока данных

```java
// 1. Controller получает HTTP запрос
@PostMapping("/calculate")
public String calculate(@Valid QuickInputDTO dto) {
    // 2. Вызывает Service
    SimulationResult result = calculationService.calculateQuick(dto);
    // 3. Service использует Repository для сохранения
    // 4. Возвращает результат
}
```

Почему это хорошо?

Принцип	Реализация
Single Responsibility	Controller не знает о БД
Testability	Service тестируется без HTTP
Maintainability	Изменения изолированы

---

3. JPA и ORM

Проблема: Impedance Mismatch

```
JAVA                          SQL
─────                         ────
Object                        Row
Field                         Column
Reference (object.field)      Foreign Key (JOIN)
Inheritance                   ???
```

Решение: ORM (Object-Relational Mapping)

JPA (Java Persistence API) — стандарт ORM в Java.
Hibernate — самая популярная реализация.

Пример Entity

```java
@Entity                              // Это JPA Entity
@Table(name = "applications")        // Имя таблицы
public class Application {

    @Id                              // Primary Key
    @GeneratedValue(strategy = GenerationType.IDENTITY)  // Auto-increment
    private Long id;

    @Column(nullable = false)        // NOT NULL
    private String email;

    @Enumerated(EnumType.STRING)     // Enum как строка
    private TaxSystem taxSystem;

    @OneToOne(mappedBy = "application")  // Связь 1:1
    private SimulationResult result;
}
```

Генерируемый SQL

```sql
CREATE TABLE applications (
    id BIGSERIAL PRIMARY KEY,
    email VARCHAR(255) NOT NULL,
    tax_system VARCHAR(10),
    ...
);
```

Стратегии генерации ID

Стратегия	Как работает	Когда использовать
`IDENTITY`	AUTO_INCREMENT в БД	MySQL, PostgreSQL
`SEQUENCE`	Sequence в БД	PostgreSQL (рекомендуется)
`TABLE`	Отдельная таблица	Переносимость
`AUTO`	Провайдер выбирает	Простота

---

4. Паттерн DTO

Проблема

```java
// Плохо: используем Entity напрямую в Controller
@PostMapping("/save")
public String save(Application application) {  // Entity!
    // Пользователь может установить id=1 и перезаписать чужие данные!
    applicationRepository.save(application);
}
```

Решение: DTO (Data Transfer Object)

```
// Хорошо: отдельный DTO для формы
@PostMapping("/save")
public String save(@Valid QuickInputDTO dto) {  // DTO!
    Application app = mapToEntity(dto);  // Мы контролируем маппинг
    applicationRepository.save(app);
}
```

Почему DTO важен?

Проблема	Решение с DTO
Mass Assignment	DTO не содержит id, createdAt
Validation	Разная валидация для форм и БД
Flexibility	Форма может иметь другую структуру
Security	Скрываем внутренние поля

Пример маппинга

```
private Application mapDtoToApplication(QuickInputDTO dto) {
    return Application.builder()
        // id генерируется автоматически
        // publicId генерируется в @PrePersist
        .industry(dto.getIndustry())
        .monthlyRevenue(dto.getMonthlyRevenue())
        .email(dto.getEmail())
        // createdAt устанавливается в @PrePersist
        .build();
}
```

---

5. Dependency Injection

Проблема: Tight Coupling

```
// Плохо: жёсткая связь
public class CalculationService {
    private ApplicationRepository repo = new ApplicationRepository(); // !!!
}
```

Проблемы:
- Невозможно тестировать с mock
- Невозможно подменить реализацию
- Нарушение DIP (Dependency Inversion Principle)

Решение: Dependency Injection

```
// Хорошо: зависимость внедряется извне
@Service
@RequiredArgsConstructor  // Lombok создаёт конструктор
public class CalculationService {

    private final ApplicationRepository repo;  // Spring внедрит!

    // Lombok генерирует:
    // public CalculationService(ApplicationRepository repo) {
    //     this.repo = repo;
    // }
}
```

Способы DI в Spring

```
// 1. Конструктор (РЕКОМЕНДУЕТСЯ)
private final Service service;

public Controller(Service service) {
    this.service = service;
}

// 2. @Autowired на поле (legacy)
@Autowired
private Service service;

// 3. Setter (редко)
@Autowired
public void setService(Service service) {
    this.service = service;
}
```

Почему конструктор лучше?

Преимущество	Объяснение
Immutability	Поле final
Required	Нельзя создать без зависимости
Testing	Легко передать mock
Clarity	Все зависимости видны

---

6. Thymeleaf и шаблоны

Почему Thymeleaf, а не React/Vue?

Thymeleaf (SSR)	React/Vue (SPA)
Простота для MVP	Сложнее настройка
SEO из коробки	Нужен SSR
Меньше кода	Больше кода
Быстрая разработка	Гибче UI

Синтаксис Thymeleaf

```
<!-- Вывод текста -->
<span th:text="${user.name}">Default Name</span>

<!-- Итерация -->
<tr th:each="item : ${items}">
    <td th:text="${item.name}"></td>
</tr>

<!-- Условия -->
<div th:if="${user != null}">Logged in</div>
<div th:unless="${user != null}">Guest</div>

<!-- Формы -->
<form th:action="@{/save}" th:object="${userForm}" method="post">
    <input th:field="*{email}" />
</form>

<!-- Фрагменты -->
<div th:replace="~{fragments/layout :: navbar}"></div>
```

Layout Pattern

```
<!-- fragments/layout.html -->
<nav th:fragment="navbar">
    <a href="/">Home</a>
    <a href="/about">About</a>
</nav>

<!-- index.html -->
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<!-- Будет вставлена навигация -->
```

---

7. Метод Monte Carlo

Что это?

Статистический метод, основанный на многократном случайном моделировании.

Как работает в BankStart?

```
1. Берём входные данные:
   - Выручка = 10,000 BYN/мес
   - Волатильность = 15%

2. Прогоняем 1000 симуляций:

   Симуляция 1: выручка колеблется случайно
   ├─ Месяц 1: 10,500 BYN (+5%)
   ├─ Месяц 2: 9,200 BYN (-8%)
   ├─ ...
   └─ Месяц 12: cash flow = +2,500 BYN ✓ (выжил)

   Симуляция 2:
   ├─ Месяц 1: 8,500 BYN (-15%)
   ├─ ...
   └─ Месяц 7: cash flow = -500 BYN ✗ (банкрот)

   ...

   Симуляция 1000: ...

3. Считаем результат:
   850 из 1000 выжили → Вероятность успеха = 85%
```

Код симуляции (упрощённо)

```java
for (int i = 0; i < ITERATIONS; i++) {
    BigDecimal cashFlow = initialCapital;

    for (int month = 1; month <= 12; month++) {
        // Случайное колебание выручки (нормальное распределение)
        double randomFactor = 1 + random.nextGaussian() * volatility;
        BigDecimal monthRevenue = revenue.multiply(BigDecimal.valueOf(randomFactor));

        // Расходы и налоги
        BigDecimal costs = fixedCosts.add(monthRevenue.multiply(variablePercent));
        BigDecimal taxes = calculateTax(monthRevenue, costs);

        // Обновляем cash flow
        cashFlow = cashFlow.add(monthRevenue).subtract(costs).subtract(taxes);
    }

    if (cashFlow.compareTo(BigDecimal.ZERO) >= 0) {
        successCount++;
    }
}

BigDecimal successProbability = successCount / ITERATIONS * 100;
```

Перцентили (P10, P50, P90)

```
P10 = 10% результатов ХУЖЕ этого значения (пессимистичный)
P50 = Медиана, 50% хуже / 50% лучше (средний)
P90 = 90% результатов ХУЖЕ этого значения (оптимистичный)

Пример (1000 симуляций, отсортировано):
Позиция 100  (10%) → P10 = -5,000 BYN (плохой сценарий)
Позиция 500  (50%) → P50 = +10,000 BYN (средний)
Позиция 900  (90%) → P90 = +25,000 BYN (хороший)
```

---

8. BigDecimal для денег

НИКОГДА не используйте double для денег!

```java
// Проблема с double
double a = 0.1;
double b = 0.2;
System.out.println(a + b);  // 0.30000000000000004 !!!

// Решение: BigDecimal
BigDecimal a = new BigDecimal("0.1");
BigDecimal b = new BigDecimal("0.2");
System.out.println(a.add(b));  // 0.3 ✓
```

Правила работы с BigDecimal

```java
// Создание
BigDecimal amount = new BigDecimal("1000.50");  // Из строки!
BigDecimal rate = BigDecimal.valueOf(0.06);     // Или valueOf

// Операции (не изменяют оригинал!)
BigDecimal result = amount.add(other);
BigDecimal result = amount.subtract(other);
BigDecimal result = amount.multiply(other);
BigDecimal result = amount.divide(other, 2, RoundingMode.HALF_UP);

// Сравнение
if (a.compareTo(b) > 0) { /* a > b */ }
if (a.compareTo(BigDecimal.ZERO) < 0) { /* a < 0 */ }

// НИКОГДА не используйте equals для сравнения!
// new BigDecimal("1.0").equals(new BigDecimal("1.00")) == false!
```

Аннотации JPA для денег

```java
@Column(precision = 15, scale = 2)  // 15 цифр всего, 2 после запятой
private BigDecimal monthlyRevenue;  // До 999,999,999,999.99
```

---

9. Bean Validation

Что это?

Стандарт Jakarta (ранее Java EE) для валидации данных через аннотации.

Основные аннотации

```
// Не null
@NotNull(message = "Поле обязательно")
private String field;

// Не пустая строка
@NotBlank(message = "Поле не должно быть пустым")
private String text;

// Числовые ограничения
@Min(value = 0, message = "Минимум 0")
@Max(value = 100, message = "Максимум 100")
private Integer percent;

// Положительные числа
@Positive(message = "Должно быть > 0")
@PositiveOrZero(message = "Должно быть >= 0")
private BigDecimal amount;

// Email
@Email(message = "Некорректный email")
private String email;

// Паттерн
@Pattern(regexp = "^\\+?[0-9]{10,15}$", message = "Некорректный телефон")
private String phone;

// Размер
@Size(min = 2, max = 100, message = "От 2 до 100 символов")
private String name;
```

Как работает валидация?

```
// В Controller добавляем @Valid
@PostMapping("/save")
public String save(@Valid @ModelAttribute QuickInputDTO dto,
                   BindingResult result) {
    if (result.hasErrors()) {
        // Есть ошибки валидации
        return "form";  // Показываем форму с ошибками
    }
    // Данные валидны
    service.save(dto);
    return "redirect:/success";
}
```

Отображение ошибок в Thymeleaf

```
<input th:field="*{email}" th:errorclass="is-invalid" />
<div class="invalid-feedback" th:errors="*{email}">
    Сообщение об ошибке появится здесь
</div>
```

---

10. Профили и конфигурация

Зачем нужны профили?

```
Development (dev):
- H2 in-memory БД
- Логи DEBUG
- Кэш отключен

Production (prod):
- PostgreSQL
- Логи WARN
- Кэш включен
```

Как работает?

```
# application.properties — общие настройки
spring.application.name=BankStart

# application-dev.properties — только для dev
spring.datasource.url=jdbc:h2:mem:bankstart

# application-prod.properties — только для prod
spring.datasource.url=jdbc:postgresql://localhost:5432/bankstart
```

Активация профиля

```
# Через аргумент
java -jar app.jar --spring.profiles.active=prod

# Через переменную окружения
SPRING_PROFILES_ACTIVE=prod java -jar app.jar
```

Externalized Configuration

Порядок приоритета (от низкого к высокому):

1. `application.properties` в JAR
2. `application-{profile}.properties`
3. Переменные окружения
4. Аргументы командной строки

```
# Переопределение через env
DB_PASSWORD=secret java -jar app.jar

# Переопределение через аргумент
java -jar app.jar --server.port=9000
```

🎓 Итого: Ключевые паттерны

Паттерн	Где используется	Зачем
Layered Architecture	Вся структура	Разделение ответственности
Repository	Data Access	Абстракция БД
DTO	Controller ↔ Service	Безопасность, гибкость
Builder	Entity, DTO	Удобное создание объектов
Dependency Injection	Все компоненты	Loose coupling, тестируемость
Template Method	Thymeleaf fragments	Переиспользование UI

© 2026 BankStart MVP. Образовательный материал.